

9. Programación orientada a objetos

9.1 Clases y objetos

La programación orientada a objetos es un paradigma de programación que se basa, como su nombre indica, en la utilización de objetos. Estos objetos también se suelen llamar instancias.

Un objeto en términos de POO no se diferencia mucho de lo que conocemos como un objeto en la vida real. Pensemos por ejemplo en un coche. Nuestro coche sería un objeto concreto de la vida real, igual que el coche del vecino, o el coche de un compañero de trabajo, o un deportivo que vimos por la calle el fin de semana pasado... Todos esos coches son objetos concretos que podemos ver y tocar.

Tanto mi coche como el coche del vecino tienen algo en común, ambos son coches. En este caso mi coche y el coche del vecino serían **instancias** (objetos) y coche (a secas) sería una **clase**. La palabra coche define algo genérico, es una abstracción, no es un coche concreto sino que hace referencia a unos elementos que tienen una serie de propiedades como matrícula, marca, modelo, color, etc.; este conjunto de propiedades se denominan **atributos** o **variables de instancia**.



Clase

Concepto abstracto que denota una serie de cualidades, por ejemplo **coche**.

Instancia

Objeto palpable, que se deriva de la concreción de una clase, por ejemplo **mi coche**.

Atributos

Conjunto de características que comparten los objetos de una clase, por ejemplo para la clase **coche** tendríamos **matrícula**, **marca**, **modelo**, **color** y **número de plazas**.

En Java, los nombres de las clases se escriben con la primera letra en mayúscula mientras que los nombres de las instancias comienzan con una letra en minúscula. Por ejemplo, la clase coche se escribe en Java como `Coche` y el objeto “mi coche” se podría escribir como `miCoche`.

Definiremos cada clase en un fichero con el mismo nombre más la extensión `.java`, por tanto, la definición de la clase `Coche` debe estar contenida en un fichero con nombre `Coche.java`

Vamos a definir a continuación la clase `Libro` con los atributos `isbn`, `autor`, `titulo` y `numeroPaginas`.

```
/**
 * Libro.java
 * Definición de la clase Libro
 * @author Luis José Sánchez
 */
public class Libro {
    // atributos
    String isbn;
    String titulo;
    String autor;
    int numeroDePaginas;
}
```

A continuación creamos varios objetos de esta clase.

```
/**
 * PruebaLibro.java
 * Programa que prueba la clase Libro
 * @author Luis José Sánchez
 */
public class PruebaLibro {
    public static void main(String[] args) {
        Libro lib = new Libro();
        Libro miLibrito = new Libro();
        Libro quijote = new Libro();
    }
}
```

Hemos creado tres instancias de la clase `libro`: `lib`, `miLibrito` y `quijote`. Ya sabemos definir una clase indicando sus atributos y crear instancias.



Las **variables de instancia** (atributos) determinan las **cualidades** de los objetos.

9.2 Encapsulamiento y ocultación

Uno de los pilares en los que se basa la Programación Orientada a Objetos es el **encapsulamiento**. Básicamente, el **encapsulamiento** consiste en definir todas las

propiedades y el comportamiento de una clase dentro de esa clase; es decir, en la clase `Coche` estará definido todo lo concerniente a la clase `Coche` y en la clase `Libro` estará definido todo lo que tenga que ver con la clase `Libro`.

El **encapsulamiento** parece algo obvio, casi de perogrullo, pero hay que tenerlo siempre muy presente al programar utilizando clases y objetos. En alguna ocasión puede que estemos tentados a mezclar parte de una clase con otra clase distinta para resolver un problema puntual. No hay que caer en esa trampa. Se deben escribir los programas de forma que cada cosa esté en su sitio. Sobre todo al principio, cuando definimos nuestras primeras clases, debemos estar pendientes de que todo está definido donde corresponde.

La **ocultación** es una técnica que incorporan algunos lenguajes (entre ellos Java) que permite esconder los elementos que definen una clase, de tal forma que desde otra clase distinta no se pueden “ver las tripas” de la primera. La **ocultación** facilita, como veremos más adelante, el **encapsulamiento**.

9.3 Métodos

Un coche arranca, para, se aparca, hace sonar el claxon, se puede llevar a reparar...
Un gato puede comer, dormir, maullar, ronronear...

Las acciones asociadas a una clase se llaman métodos. Estos métodos se definen dentro del cuerpo de la clase y se suelen colocar a continuación de los atributos.



Los **métodos** determinan el **comportamiento** de los objetos.

Creí haber visto un lindo gatito

Vamos a crear la clase `GatoSimple`. La llamamos así porque más adelante crearemos otra clase algo más elaborada que se llamará `Gato`. Para saber qué atributos debe tener esta clase hay que preguntarse qué características tienen los gatos. Todos los gatos son de un color determinado, pertenecen a una raza, tienen una edad, tienen un determinado sexo - son machos o hembras - y tienen un peso que se puede expresar en kilogramos. Éstos serán por tanto los atributos que tendrá la clase `GatoSimple`.

Para saber qué métodos debemos implementar hay que preguntarse qué acciones están asociadas a los gatos. Bien, pues los gatos maullan, ronronean, comen y si son machos se pelean entre ellos para disputarse el favor de las hembras. Esos serán los métodos que definamos en la clase.

```
/**
 * GatoSimple.java
 * Definición de la clase GatoSimple
 * @author Luis José Sánchez
 */
public class GatoSimple {

    // atributos //////////////////////////////////////

    String color, raza, sexo;
    int edad;
    double peso;

    // métodos //////////////////////////////////////

    // constructor
    GatoSimple (String s) {
        this.sexo = s;
    }

    // getter
    String getSexo() {
        return this.sexo;
    }

    /**
     * Hace que el gato maulle
     */
    void maulla() {
        System.out.println("Miauuuu");
    }

    /**
     * Hace que el gato ronronee
     */
    void ronronea() {
        System.out.println("mrrrrrrr");
    }

    /**
     * Hace que el gato coma.
     * A los gatos les gusta el pescado, si le damos otra comida
     * la rechazará.
     *
     * @param comida la comida que se le ofrece al gato
     */
}
```

```

void come(String comida) {
    if (comida.equals("pescado")) {
        System.out.println("Hmmm, gracias");
    } else {
        System.out.println("Lo siento, yo solo como pescado");
    }
}

/**
 * Pone a pelear dos gatos.
 * Solo se van a pelear dos machos entre sí.
 *
 * @param contrincante es el gato contra el que pelear
 */
void peleaCon(GatoSimple contrincante) {
    if (this.sexo.equals("hembra")) {
        System.out.println("no me gusta pelear");
    } else {
        if (contrincante.getSexo().equals("hembra")) {
            System.out.println("no peleo contra gatitas");
        } else {
            System.out.println("ven aquí que te vas a enterar");
        }
    }
}
}

```



Constructor

El método constructor tiene siempre el mismo nombre que la clase y se utiliza normalmente para inicializar los atributos.

Los atributos de la clase `GatoSimple` - color, raza, sexo, edad y peso - se declaran igual que las variables que hemos venido usando hasta ahora, pero hay una gran diferencia entre estos atributos y las variables que aparecen en el `main` (programa principal). Una variable definida en el cuerpo del programa principal es única, sin embargo cada uno de los objetos que se crean en el programa principal tienen sus propios atributos; es decir, si en el programa principal se crean 20 objetos de la clase `GatoSimple`, cada uno tiene sus valores para los atributos `color`, `raza`, etc.

Fíjate en la cabecera del método `peleaCon`:

```
void peleaCon(GatoSimple contrincante)
```

Como puedes comprobar, es posible pasar un objeto como parámetro. A continuación

se muestra un programa que prueba la clase `GatoSimple`. Te recomiendo seguir el programa línea a línea y observar atentamente la salida que produce.

```
/**
 * PruebaGatoSimple.java
 * Programa que prueba la clase GatoSimple
 * @author Luis José Sánchez
 */
public class PruebaGatoSimple {
    public static void main(String[] args) {

        GatoSimple garfield = new GatoSimple("macho");

        System.out.println("hola gatito");
        garfield.maulla();
        System.out.println("toma tarta");
        garfield.come("tarta selva negra");
        System.out.println("toma pescado, a ver si esto te gusta");
        garfield.come("pescado");

        GatoSimple tom = new GatoSimple("macho");

        System.out.println("Tom, toma sopita de verduras");
        tom.come("sopa de verduras");

        GatoSimple lisa = new GatoSimple("hembra");

        System.out.println("gatitos, a ver cómo maulláis");
        garfield.maulla();
        tom.maulla();
        lisa.maulla();

        garfield.peleaCon(lisa);
        lisa.peleaCon(tom);
        tom.peleaCon(garfield);
    }
}
```

Observa cómo al crear una instancia se llama al constructor que, como decíamos antes, tiene el mismo nombre de la clase y sirve para inicializar los atributos. En este caso se inicializa el atributo `sexo`. Más adelante veremos constructores que inicializan varios atributos e incluso definiremos distintos constructores en la misma clase.

```
GatoSimple garfield = new GatoSimple("macho");
```

Como puedes ver hay métodos que no toman ningún parámetro.

```
garfield.maula();
```

Y hay otros métodos que deben tomar parámetros obligatoriamente.

```
garfield.come("tarta selva negra");
```

Métodos *getter* y *setter*

Vamos a crear la clase `Cubo`. Para saber qué atributos se deben definir, nos preguntamos qué características tienen los cubos - igual que hicimos con la clase `GatoSimple`. Todos los cubos tienen una determinada capacidad, un color, están hechos de un determinado material - plástico, latón, etc. - y puede que tengan asa o puede que no. Un cubo se fabrica con el propósito de contener líquido; por tanto otra característica es la cantidad de litros de líquido que contiene en un momento determinado. Por ahora, solo nos interesa saber la capacidad máxima y los litros que contiene el cubo en cada momento, así que esos serán los atributos que tendremos en cuenta.

```
/**
 * Cubo.java
 * Definición de la clase Cubo
 * @author Luis José Sánchez
 */
public class Cubo {

    // atributos //////////////////////////////////////

    int capacidad; // capacidad máxima en litros
    int contenido; // contenido actual en litros

    // métodos //////////////////////////////////////

    // constructor
    Cubo (int c) {
        this.capacidad = c;
    }

    // métodos getter
    int getCapacidad() {
        return this.capacidad;
    }

    int getContenido() {
        return this.contenido;
    }
}
```

```
// método setter
void setContenido(int litros) {
    this.contenido = litros;
}

// otros métodos
void vacia() {
    this.contenido = 0;
}

/**
 * Llena el cubo al máximo de su capacidad.
 */
void llena() {
    this.contenido = this.capacidad;
}

/**
 * Pinta el cubo en la pantalla.
 * Se muestran los bordes del cubo con el carácter # y el
 * agua que contiene con el carácter ~.
 */
void pinta() {
    for (int nivel = this.capacidad; nivel > 0; nivel--) {
        if (this.contenido >= nivel) {
            System.out.println("#~~~~#");
        } else {
            System.out.println("#    #");
        }
    }
    System.out.println("#####");
}

/**
 * Vuelca el contenido de un cubo sobre otro.
 * Antes de echar el agua se comprueba cuánto le cabe al
 * cubo destino.
 */
void vuelcaEn(Cubo destino) {
    int libres = destino.getCapacidad() - destino.getContenido();

    if (libres > 0) {
        if (this.contenido <= libres) {
            destino.setContenido(destino.getContenido() + this.contenido);
            this.vacia();
        }
    }
}
```

```
    } else {  
        this.contenido -= libres;  
        destino.llena();  
    }  
}  
}
```

Observa estos métodos extraídos de la clase `Cubo`:

```
int getCapacidad() {  
    return this.capacidad;  
}  
  
int getContenido() {  
    return this.contenido;  
}
```

Se trata de métodos muy simples, su cometido es devolver el valor de un atributo. Podrían tener cualquier nombre pero en Java es costumbre llamarlos con la palabra *get* (obtener) seguida del nombre del atributo.

Fijémonos ahora este otro método:

```
void setContenido(int litros) {  
    this.contenido = litros;  
}
```

Ahora estamos ante un *setter*. Este tipo de métodos tiene el cometido de establecer un valor para un determinado atributo. Como puedes ver, `setContenido` está formada por *set* (asignar) más el nombre del atributo.

Podrías preguntarte: ¿por qué se crea un método *getter* para extraer el valor de una variable y no se accede a la variable directamente?

Parece más lógico hacer esto

```
System.out.print(miCubo.capacidad);
```

que esto otro

```
System.out.print(miCubo.getCapacidad());
```

Sin embargo, en Java se opta casi siempre por esta última opción. Tiene su explicación y lo entenderás bien cuando estudies el apartado [Ámbito/visibilidad de los elementos de una clase - public , protected y private](#)

Probamos, con el siguiente ejemplo, la clase `Cubo` que acabamos de definir. Crea tus propios cubos, pasa agua de unos a otros y observa lo que se muestra por pantalla.

```
/**
 * PruebaCubo.java
 * Programa que prueba la clase Cubo
 * @author Luis José Sánchez
 */
public class PruebaCubo {
    public static void main(String[] args) {

        Cubo cubito = new Cubo(2);
        Cubo cubote = new Cubo(7);

        System.out.println("Cubito: \n");
        cubito.pinta();

        System.out.println("\nCubote: \n");
        cubote.pinta();

        System.out.println("\nLleno el cubito: \n");
        cubito.llena();
        cubito.pinta();

        System.out.println("\nEl cubote sigue vacío: \n");
        cubote.pinta();

        System.out.println("\nAhora vuelco lo que tiene el cubito en el cubote.\n");
        cubito.vuelcaEn(cubote);

        System.out.println("Cubito: \n");
        cubito.pinta();

        System.out.println("\nCubote: \n");
        cubote.pinta();

        System.out.println("\nAhora vuelco lo que tiene el cubote en el cubito.\n");
        cubote.vuelcaEn(cubito);

        System.out.println("Cubito: \n");
        cubito.pinta();

        System.out.println("\nCubote: \n");
        cubote.pinta();
    }
}
```

Método toString

La definición de la clase `Cubo` del apartado anterior contiene el método `pinta` que, como su nombre indica, permite pintar en pantalla un objeto de esa clase. Se podría definir el método `ficha` para mostrar información sobre objetos de la clase `Alumno` por ejemplo. También sería posible implementar el método `imprime` dentro de la clase `Libro` con el propósito de mostrar por pantalla los datos de un libro. Todos estos métodos - `pinta`, `ficha` e `imprime` - hacen básicamente lo mismo.

En Java existe una solución muy elegante para mostrar información sobre un objeto por pantalla. Si se quiere mostrar el contenido de la variable entera `x` se utiliza `System.out.print(x)` y si se quiere mostrar el valor de la variable de tipo cadena de caracteres `nombre` se escribe `System.out.print(nombre)`. De la misma manera, si se quiere mostrar el objeto `miPiramide` que pertenece a la clase `Piramide`, también se podría usar `System.out.print(miPiramide)`. Java sabe perfectamente cómo mostrar números y cadenas de caracteres pero no sabe *a priori* cómo se pintan pirámides. Para indicar a Java cómo debe pintar un objeto de la clase `Piramide` basta con implementar el método `toString` dentro de la clase.

Veamos un ejemplo muy sencillo de implementación de `toString`. Definiremos la clase `Cuadrado` con el atributo `lado`, el constructor y el método `toString`.

```
/**
 * Cuadrado.java
 * Definición de la clase Cuadrado
 * @author Luis José Sánchez
 */
public class Cuadrado {

    int lado;

    public Cuadrado(int l) {
        this.lado = l;
    }

    public String toString() {

        int i, espacios;
        String resultado = "";

        for (i = 0; i < this.lado; i++) {
            resultado += "□□";
        }
        resultado += "\n";

        for (i = 1; i < this.lado - 1; i++) {
            resultado += "□□";
        }
    }
}
```

```

        for (espacios = 1; espacios < this.lado - 1; espacios++) {
            resultado += " ";
        }
        resultado += "□□\n";
    }

    for (i = 0; i < this.lado; i++) {
        resultado += "□□";
    }
    resultado += "\n";

    return resultado;
}
}

```

Observa que el método `toString()` devuelve una cadena de caracteres. Esa cadena es precisamente la que mostrará por pantalla `System.out.print()`. En el programa que se muestra a continuación y que prueba la clase `Cuadrado` puedes comprobar que se pinta un cuadrado igual que si se tratara de cualquier otra variable.

```

/**
 * PruebaCuadrado.java
 * Programa que prueba la clase Cuadrado
 * @author Luis José Sánchez
 */
public class PruebaCuadrado {
    public static void main(String[] args) {

        Cuadrado miCuadradito = new Cuadrado(5);
        System.out.println(miCuadradito);
    }
}

```

9.4 Ámbito/visibilidad de los elementos de una clase

▪ **public**, **protected** y **private**

Al definir los elementos de una clase, se pueden especificar sus ámbitos (*scope*) de visibilidad o accesibilidad con las palabras reservadas `public` (público), `protected` (protegido) y `private` (privado).

En la siguiente tabla se muestra desde dónde es visible/accesible un elemento (atributo o método) según el modificador que lleve.

	En la misma clase	En el mismo paquete	En una subclase	Fuera del paquete
<code>private</code>	✓	✗	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓
sin especificar	✓	✓	✗	✗

Figura 9.1: Ámbito de los elementos de una clase.

Por ejemplo, un atributo `private` solamente es accesible desde la misma clase, sin embargo, a un método `protected` se podrá acceder desde la misma clase donde esté definido, desde otro fichero dentro del mismo paquete o desde una subclase.

Como regla general, se suelen declarar `private` los atributos o variables de instancia y `public` los métodos.

En el siguiente apartado veremos qué son las subclases. De momento fíjate en el ámbito del atributo y de los métodos en la definición de la clase `Animal`.

```
/**
 * Animal.java
 * Definición de la clase Animal
 * @author Luis José Sánchez
 */
public abstract class Animal {

    private Sexo sexo;

    public Animal () {
        sexo = Sexo.MACHO;
    }

    public Animal (Sexo s) {
        sexo = s;
    }
}
```

```
}

public Sexo getSexo() {
    return sexo;
}

public String toString() {
    return "Sexo: " + this.sexo + "\n";
}

/**
 * Hace que el animal se eche a dormir.
 */
public void duerme() {
    System.out.println("Zzzzzzz");
}
}
```

Fíjate que el atributo `sexo` se ha definido como `private`.

```
private Sexo sexo;
```

Eso quiere decir que a ese atributo únicamente se tiene acceso dentro de la clase `Animal`. Sin embargo, todos los métodos se han definido `public`, lo que significa que se podrán utilizar desde cualquier otro programa, por ejemplo, como veremos más adelante, desde el programa `PurebaAnimal.java`.



Salvo casos puntuales se seguirá la regla de declarar `private` las variables de instancia y `public` los métodos.

El sexo de un animal solo puede ser macho, hembra o hermafrodita. Una forma de delimitar los valores que puede tomar un atributo es definir un tipo enumerado.



Tipo enumerado

Mediante `enum` se puede definir un tipo enumerado, de esta forma un atributo solo podrá tener uno de los posibles valores que se dan como opción. Los valores que se especifican en el tipo enumerado se suelen escribir con todas las letras en mayúscula.

9.9 Ejercicios

Conceptos de POO



Ejercicio 1

¿Cuáles serían los atributos de la clase `PilotoDeFormula1`? ¿Se te ocurren algunas instancias de esta clase?



Ejercicio 2

A continuación tienes una lista en la que están mezcladas varias clases con instancias de esas clases. Para ponerlo un poco más difícil, todos los elementos están escritos en minúscula. Di cuáles son las clases, cuáles las instancias, a qué clase pertenece cada una de estas instancias y cuál es la jerarquía entre las clases: paula, goofy, gardfiel, perro, mineral, caballo, tom, silvestre, piritá, rocinante, milu, snoopy, gato, pluto, animal, javier, bucefalo, pegaso, ayudante_de_santa_claus, cuarzo, laika, persona, pato_lucas.



Ejercicio 3

¿Cuáles serían los atributos de la clase `Vivienda`? ¿Qué subclases se te ocurren?



Ejercicio 4

Piensa en la liga de baloncesto, ¿qué 5 clases se te ocurren para representar 5 elementos distintos que intervengan en la liga?



Ejercicio 5

Haz una lista con los atributos que podría tener la clase `caballo`. A continuación haz una lista con los posibles métodos (acciones asociadas a los caballos).



Ejercicio 6

Lista los atributos de la clase `Alumno` ¿Sería `nombre` uno de los atributos de la clase? Razona tu respuesta.



Ejercicio 7

¿Cuáles serían los atributos de la clase `Ventana` (de ordenador)? ¿cuáles serían los métodos? Piensa en las propiedades y en el comportamiento de una ventana de cualquier programa.

POO en Java



Ejercicio 1

Implementa la clase `Caballo` vista en un ejercicio anterior. Pruébala creando instancias y aplicándole algunos métodos.



Ejercicio 2

Crea la clase `Vehiculo`, así como las clases `Bicicleta` y `Coche` como subclases de la primera. Para la clase `Vehiculo`, crea los atributos de clase `vehiculosCreados` y `kilometrosTotales`, así como el atributo de instancia `kilometrosRecorridos`. Crea también algún método específico para cada una de las subclases. Prueba las clases creadas mediante un programa con un menú como el que se muestra a continuación:

```
VEHÍCULOS
=====
1. Anda con la bicicleta
2. Haz el caballito con la bicicleta
3. Anda con el coche
4. Quema rueda con el coche
5. Ver kilometraje de la bicicleta
6. Ver kilometraje del coche
7. Ver kilometraje total
8. Salir
Elige una opción (1-8):
```



Ejercicio 3

Crea las clases `Animal`, `Mamifero`, `Ave`, `Gato`, `Perro`, `Canario`, `Pinguino` y `Lagarto`. Crea, al menos, tres métodos específicos de cada clase y redefine el/los método/s cuando sea necesario. Prueba las clases creadas en un programa en el que se instancien objetos y se les apliquen métodos.



Ejercicio 4

Crea la clase `Fracción`. Los atributos serán `numerador` y `denominador`. Y algunos de los métodos pueden ser `invierte`, `simplifica`, `multiplica`, `divide`, etc.



Ejercicio 5

Crea la clase `Pizza` con los atributos y métodos necesarios. Sobre cada pizza se necesita saber el tamaño - mediana o familiar - el tipo - margarita, cuatro quesos o funghi - y su estado - pedida o servida. La clase debe almacenar información sobre el número total de pizzas que se han pedido y que se han servido. Siempre que se crea una pizza nueva, su estado es "pedida". El siguiente código del programa principal debe dar la salida que se muestra:

```
public class PedidosPizza {
    public static void main(String[] args) {
        Pizza p1 = new Pizza("margarita", "mediana");
        Pizza p2 = new Pizza("funghi", "familiar");
        p2.sirve();
        Pizza p3 = new Pizza("cuatro quesos", "mediana");
        System.out.println(p1);
        System.out.println(p2);
        System.out.println(p3);
        p2.sirve();
        System.out.println("pedidas: " + Pizza.getTotalPedidas());
        System.out.println("servidas: " + Pizza.getTotalServidas());
    }
}
```

```
pizza margarita mediana, pedida
pizza funghi familiar, servida
pizza cuatro quesos mediana, pedida
esa pizza ya se ha servido
pedidas: 3
servidas: 1
```



Ejercicio 6

Crea la clase `Tiempo` con los métodos `suma` y `resta`. Los objetos de la clase `Tiempo` son intervalos de tiempo y se crean de la forma `Tiempo t = new Tiempo(1, 20, 30)` donde los parámetros que se le pasan al constructor son las horas, los minutos y los segundos respectivamente. Crea el método `toString` para ver los intervalos de tiempo de la forma `10h 35m 5s`. Si se suman por ejemplo `30m 40s` y `35m 20s` el resultado debería ser `1h 6m 0s`. Realiza un programa de prueba para comprobar que la clase funciona bien.



Ejercicio 7

Queremos gestionar la venta de entradas (no numeradas) de **Expocoches Campanillas** que tiene 3 zonas, la sala principal con 1000 entradas disponibles, la zona de compra-venta con 200 entradas disponibles y la zona vip con 25 entradas disponibles. Hay que controlar que existen entradas antes de venderlas.

La clase Zona con sus atributos y métodos se muestra a continuación:

```
public class Zona {

    private int entradasPorVender;

    public Zona(int n){
        entradasPorVender = n;
    }

    public int getEntradasPorVender() {
        return entradasPorVender;
    }

    /**
     * Vende un número de entradas.
     * Comprueba si quedan entradas libres antes de realizar la venta.
     *
     * @param n número de entradas a vender
     */
    public void vender(int n) {

        if (this.entradasPorVender == 0) {
            System.out.println("Lo siento, las entradas para esa zona están agotadas.");
        } else if (this.entradasPorVender < n) {
            System.out.println("Sólo me quedan " + this.entradasPorVender
                               + " entradas para esa zona.");
        }

        if (this.entradasPorVender >= n) {
            entradasPorVender -= n;
            System.out.println("Aquí tiene sus " + n + " entradas, gracias.");
        }
    }
}
```

El menú del programa debe ser el que se muestra a continuación. Cuando elegimos la opción 2, se nos debe preguntar para qué zona queremos las entradas y cuántas queremos. Lógicamente, el programa debe controlar que no se puedan vender más entradas de la cuenta.

1. Mostrar número de entradas libres
2. Vender entradas
3. Salir